



EDIÇÃO DE LUXO · ultra-micro

A Pizzaria do Seu Nemotron

O job **pedacinho por pedacinho** — com os personagens falando

Recapitulando o elenco: **Seu Nemotron** (chef 30B) · **Vovó** (o molho 086) · **Forno** (GPU) · **Massa Pronta** (wheel) · **Inspetor Sherllambda** (watchdog) · **Câmera** (live-log).

1 🍃 Os ingredientes — desmontando cada pedaço

📖 Abre a cortina: o mercado ainda fechado, o Seu Nemotron bate o pé impaciente...



Seu Nemotron

Sem a apostila de receitas eu NEM coloco o avental, viu?

```
subprocess.run(['git', 'clone', '--depth', '1', '--branch', 'claude/v1244-cot-safe', URL, '/content/kg1'], check=
```

🔬 Dissecando pedaço por pedaço:

Pedaço	O que é (técnico)	Quem é na pizzeria 🍕
<code>subprocess</code>	módulo que roda programas de fora do Python	o telefone pra ligar pro mercado
<code>.run(...)</code>	executa E ESPERA terminar	"faz o pedido e aguarda chegar"
<code>'git'</code>	o programa a chamar	o entregador (o Sr. Git 🚚)
<code>'clone'</code>	comando: copiar um repositório	"me traz uma cópia da apostila"
<code>'--depth', '1'</code>	baixa só o último commit	"só a última página, não a saga toda"
<code>'--branch', 'claude/...'</code>	qual ramo do repo	"a EDIÇÃO certa da apostila"
<code>'/content/kg1'</code>	pasta de destino	a prateleira onde guarda
<code>check=True</code>	levanta erro se o git falhar	"não chegou? FECHA a loja agora"

```
subprocess.run([ ... 'torch==2.10.0', '--index-url', 'https://download.pytorch.org/whl/cu126' ])
```



Inspetor Sherllambda

O Colab entregou a batedeira 2.11... mas a massa pronta só encaixa na 2.10. Caso encerrado: a gente TROCA.

🔬 Dissecando:

Pedaço	Técnico	Na pizzeria 🍕
<code>'torch==2.10.0'</code>	fixa a versão exata do torch	"quero a batedeira modelo 2.10, nem mais nem menos"
<code>'==' (dois iguais)</code>	versão EXATA (não >=)	"esse modelo e ponto final"
<code>--index-url .../cu126</code>	baixa da loja oficial do PyTorch (com CUDA)	a loja que entrega batedeira COM MOTOR ⚡
<code>cu126</code>	build pra CUDA 12.6	o tipo de tomada/motor

⚠️ CUIDADO! (piada séria)

Comprar torch na loja errada = batedeira **sem motor (CPU)**. Ela liga, acende a luzinha, faz "vrum" ... e **não bate nada**. Você só descobre quando a massa continua dura. 🤪

```
tmm = '.'.join(torch.__version__.split('+')[0].split('.')[0:2]) # vira "2.10"
```

🔬 Dissecando a linha mais "code-golf" do job:

Pedaço	O que faz	Exemplo
<code>torch.__version__</code>	a string da versão	"2.10.0+cu126"
<code>.split('+')[0]</code>	corta no "+" e pega o 1º pedaço	"2.10.0"
<code>.split('.')</code>	quebra nos pontos	["2", "10", "0"]
<code>[0:2]</code>	pega os 2 primeiros	["2", "10"]

`'.'.join(...)`

cola com ponto

`"2.10"` ✓

Nos bastidores

Por que tanto malabarismo só pra virar "2.10"? Porque é com esse "2.10" que a gente ****monta o endereço da massa pronta**** (...torch2.10...). Um caractere errado aqui = massa pronta não encontrada = 30 min sovando. Detalhe é tudo na cozinha!

```
if subprocess.run([ ... '--no-deps', url]).returncode != 0: # a massa pronta falhou?
    ... compila do source                                     # plano B: sova do zero
```

Pedaço	Técnico	Na pizzaria 🍕
<code>--no-deps</code>	instala SÓ esse pacote, sem puxar dependências	"só a massa, sem vir tralha de brinde"
<code>.returncode</code>	0 = deu certo; ≠0 = falhou	o "joinha" ou "joia pra baixo" do entregador
<code>if ... != 0:</code>	se falhou, entra no plano B	"acabou a massa pronta? bota a mão na massa"

 **Resumo Cél 1:** ligou pro mercado (git) → trocou a batedeira (torch 2.10 com motor) → leu a etiqueta ("2.10") → pegou a massa pronta (10s) ou sovou (plano B) → provou → DEPS OK.

2 A chave do depósito — desmontada

```
for k in ['HF_KEY', 'HF_TOKEN', 'HUGGINGFACE_TOKEN']:
    v = userdata.get(k)
    if v: os.environ['HF_TOKEN'] = v; break
```

Dissecando:

Pedaço	Técnico	Na pizzaria 🍕
<code>for k in [3 nomes]</code>	tenta 3 nomes de chave	testa 3 chaveiros (cada um chama de um jeito)
<code>userdata.get(k)</code>	lê o segredo do cofre do Colab	abre o cofrinho de senhas
<code>if v: ... break</code>	achou? guarda e PARA	"achei a chave! pode parar de procurar"



Vovó (o molho 086)

Sem a chavinha do depósito, ninguém pega meu molho secreto, meu bem. 🤖

3

 Medir o forno — desmontada

```
vram = torch.cuda.get_device_properties(0).total_memory / 1e9
assert vram ≥ 70, '40GB → RECONECTE p/ 80GB'
```

 Dissecando:

Pedaço	Técnico	Na pizzaria 
<code>get_device_properties(0)</code>	dados da GPU nº 0	a ficha técnica do forno
<code>.total_memory</code>	VRAM total em bytes	o tamanho do forno (em bytes)
<code>/ 1e9</code>	bytes → gigabytes	converte pra "GB" de gente
<code>assert vram>=70</code>	exige ≥70GB, senão erro	"forno < 70GB? PARA, pizza família não cabe"

**Inspetor Sherllambda**

42GB? Elementar: a pizza de 60GB NÃO entra. Eu te paro agora — antes do incêndio, não depois.  

 O "erro" aqui é HERÓI

Quando aparece `AssertionError: VRAM=42GB`, NÃO é bug — é o Inspetor te salvando de um OOM que só explodiria 10 min depois. Detector de fumaça apitando = casa salva.

4

 A ficha da masterclass — os pedaços que importam

```
os.environ['MAX_STEPS']='160'; os.environ['NUM_EPOCHS']='6' # o pulo do gato
```

 A conta que quase deu ruim (dissecada):

Pedaço	Conta	Resultado
979 receitas / 32 por panela	$\text{ceil}(979/32)$	31 passos por turma
<code>NUM_EPOCHS = 6</code>	31×6	186 passos planejados
<code>MAX_STEPS = 160</code>	$\text{min}(186, 160)$	corta em 160 
(se <code>epochs=1</code>)	$\text{min}(31, 160)$	só 31!  a armadilha

**Seu Nemotron**

Se eu fizer só 31 passos achando que são 160, saio achando que treinei e... mal aqueci. Por isso 6 turmas!

```
r = subprocess.run([sys.executable, 'scripts/kg1_colab_realtime_runner.py', '--', 'python', 'scripts/hf_job_
```

Dissecando a "claquete" do treino:

Pedaço	Técnico	Na pizzaria 🍕
<code>sys.executable</code>	o Python certo	o mesmo cozinheiro de sempre
<code>kg1_colab_realtime_runner.py</code>	o WRAPPER (vigia)	o Inspetor 🕵️ + a Câmera 📹
<code>'--'</code>	separador: fim das opções do vigia	"daqui pra frente é o COZINHEIRO que eu vigio"
<code>'python' ...train_v90.py</code>	o comando VIGIADO (o chef cozinhando)	Seu Nemotron fazendo a pizza 🍕
<code>r = ...</code>	guarda o resultado	o boletim do prato no fim



Inspetor Sherllambda

Tudo depois do `--` é o suspeito que EU vigio. Câmera ligada, 45s de heartbeat, e se sentir cheiro de NaN/OOM... desligo o fogão. 🚒

```
print('RETURN_CODE=', r.returncode) # 0 = pizza perfeita; outro = deu ruim
```

 **Fim do episódio:** → a pizza saiu, o crítico (Claude) aplaude. Qualquer outro número → o Inspetor já filmou onde pegou fogo.

Cena pós-créditos

A versão "linha por linha" (sem dissecar token) está no `JOB_LINHA_A_LINHA_USE_A_CABECA.md`. A história completa, no `A_PIZZARIA_DO_SEU_NEMOTRON.pdf`. Esta aqui é o ****director's cut****: cada byte com legenda e os personagens falando. 🎬